

| HIGH LEVEL DESIGN DOCUMENT

CredVault

A Microservices-Based Credit Card Management
& Payment Platform

Angular 19

.NET 10

Ocelot Gateway

RabbitMQ + MassTransit

SQL Server

Redis

Clean Architecture

CQRS + MediatR

DOCUMENT TYPE

HLD

VERSION

1.0

YEAR

2026

Table of Contents

CredVault High Level Design · 10 Sections

1	System Overview & Purpose	
2	Architecture Diagram	
3	Technology Stack & Rationale	
4	Component / Service Breakdown	
5	Data Flow	
6	API Contract Overview	
7	Security Approach	
8	Scalability & Reliability Strategy	
9	Assumptions & Constraints	

System Overview & Purpose

CredVault is a comprehensive credit card management platform that consolidates multiple cards, billing cycles, payments, and rewards into a single, secure, and intelligent interface — built on a modern microservices foundation.

The system is architected for modularity, scalability, and independent service evolution. Every core concern — identity, cards, billing, payments, and notifications — is owned by a dedicated microservice, communicating either synchronously via HTTP (through an API Gateway) or asynchronously via a RabbitMQ event bus.

Core Purpose



Consolidated Card Management

Provide a single dashboard to track multiple credit cards, balances, and utilization across providers.



Automated Billing Tracking

Automate billing cycle monitoring, due date reminders, and payment status updates without manual input.



Secure Payment Workflow

Execute payments through a risk-evaluated saga with real-time fraud detection and OTP verification for high-value transactions.



Rewards Engagement

Automatically compute and credit reward points on every successful payment, with tier-based loyalty tracking.



Real-time Notifications

Deliver event-driven email communications for OTPs, payment receipts, and billing reminders via template-based emails.



Audit Logging

Maintain a central, append-only audit log of all domain events for traceability, compliance, and debugging.

Architecture Diagram

All client traffic flows through a single **Ocelot API Gateway**, which handles JWT validation, request routing, and CorrelationId injection. Services communicate via HTTP (synchronous) for user-facing requests and RabbitMQ (asynchronous) for domain events.

HIGH-LEVEL SYSTEM ARCHITECTURE



EVENT PUBLISHERS & CONSUMERS

Publishers

Identity → OTPRequested, UserRegistered
Card → CardAdded, CardRemoved
Billing → BillGenerated, BillOverdue

Consumers

Card → PaymentCompleted (update balance)
Billing → PaymentCompleted (mark paid, rewards)
Notification → All events (email + audit log)

Payment → PaymentInitiated, PaymentCompleted,
FraudDetected

Technology Stack & Rationale

Component	Technology	Rationale
Frontend	Angular 19	High performance with Signals, standalone components for modularity, and strong typing via TypeScript.
Backend	.NET 10	Enterprise-grade performance, robust ecosystem, and first-class support for microservices and async patterns.
API Gateway	Ocelot	Lightweight, configuration-driven routing with built-in JWT validation, rate limiting, and load balancing.
Messaging	RabbitMQ + MassTransit	MassTransit provides high-level abstractions for Pub/Sub and complex Sagas (state machines) on top of RabbitMQ.
Database	SQL Server	Reliable relational storage for transactional integrity, per-service isolation, and complex queries.
Caching	Redis	Ultra-fast in-memory store for transient data like OTP codes (5-min TTL) and JWT blacklists.
Architecture	Clean Architecture	Enforces strict separation: Domain → Application → Infrastructure → API. Maximizes testability.
Pattern	CQRS + MediatR	Decouples read (Queries) from write (Commands) operations, simplifying complex business logic handlers.

Component / Service Breakdown

Each microservice is an independently deployable unit with its own database, owning exactly one bounded context. Internal implementation follows Clean Architecture (Domain → Application → Infrastructure → API).



Identity Service /api/identity

Role: User Identity & Access Management (IAM). Handles registration with BCrypt (factor 12) password hashing, JWT + Refresh Token issuance, and OTP generation for MFA. OTPs are stored in Redis with a 5-minute TTL and invalidated after use.



Card Service /api/cards

Role: Credit card lifecycle management. Supports add/remove, auto-detects card issuers via BIN lookup, tracks outstanding balances, monitors card expiry dates, and subscribes to PaymentCompleted events to update real-time balances.



Billing Service /api/billing

Role: Billing cycle orchestration and loyalty management. Handles monthly bill generation, tracks payment status (Pending → Paid → Overdue), schedules future payments, and manages reward point accounts and tier progression on PaymentCompleted events.



Payment Service /api/payments

Role: Payment execution engine. Runs a MassTransit Saga (state machine) for every transaction. Performs real-time risk scoring and fraud detection. Generates an immutable transaction log. Publishes PaymentCompleted or compensation (reversal) events based on outcome.



Notification Service /api/notify

Role: System-wide auditor and communicator. Pure consumer service — subscribes to all domain events, dispatches template-based HTML emails via SendGrid, and maintains a central append-only Audit Log for every significant system event.

Data Flow

5.1 — Synchronous Request Flow

1

User Action

The Angular SPA sends an HTTP request with a JWT Bearer token in the Authorization header.

2

Gateway Processing

Ocelot validates the JWT signature and claims, injects a `CorrelationId` header, applies rate limiting, and routes the request to the correct downstream service.

3

Service Execution

The target service processes the request via a MediatR Command (write) or Query (read) handler. The handler interacts with the service-owned SQL Server database and returns the response.

5.2 — Asynchronous Event Flow

1

Trigger

A service successfully completes a state change — e.g., Payment Service marks a payment as completed.

2

Publish

The service publishes a typed domain event (e.g., `PaymentCompleted`) to RabbitMQ via MassTransit — fire and forget, non-blocking.

3

Consume

All subscribed consumers (Billing, Notification, Card) receive the event from their respective queues and execute their local side-effects — update bill status, send receipt email, update balance — independently and in parallel.

5.3 — Payment Saga Flow

PAYMENT STATE MACHINE (MASTRANSIT SAGA)

User initiates payment



PaymentInitiated published



Risk Score evaluated

RISK DECISION — THREE PATHS

● Low Risk



Process immediately



PaymentCompleted ✓

● Medium Risk



Suspend → OTP verify



Resume → Completed ✓

● High Risk



Block payment



FraudDetected + Compensation ✗

Saga Guarantee: MassTransit persists the Saga state in the database. If the service restarts mid-flow, the saga resumes from exactly where it left off — ensuring no transaction is permanently lost.

API Contract Overview

All endpoints are exposed through the Ocelot Gateway. The table below shows the surface-level API contract for each service. Detailed request/response schemas are defined in the LLD document.

Service	Base Route	Key Endpoints			
Identity	<code>/api/identity</code>	POST <code>/register</code>	POST <code>/login</code>	POST <code>/refresh-token</code>	
		POST <code>/send-otp</code>	POST <code>/verify-otp</code>		
Card	<code>/api/cards</code>	GET <code>/</code>	POST <code>/add</code>	DELETE <code>/ {id}</code>	PUT <code>/default/{id}</code>
		GET <code>/utilization</code>			
Billing	<code>/api/billing</code>	GET <code>/bills</code>	GET <code>/bills/{id}</code>	POST <code>/schedule</code>	GET <code>/rewards</code>
Payment	<code>/api/payments</code>	POST <code>/initiate</code>	GET <code>/history</code>	POST <code>/verify-payment-otp</code>	
		GET <code>/risk-score</code>			
Notification	<code>/api/notify</code>	GET <code>/logs</code>	GET <code>/audit</code>		

Security Approach

CredVault implements a **defense-in-depth security model** — multiple independent layers, so a breach of one layer does not compromise the entire system.

Authentication — JWT (Stateless)



JSON Web Tokens for stateless authentication. Tokens include Claims (UserId, Email, Roles) signed with HMAC-SHA256. Stored in JavaScript memory only — never in localStorage or cookies — eliminating XSS token theft risk.

Multi-Factor Authentication (MFA)



OTP-based verification required for sensitive actions — login and high-value payments. OTPs are 6-digit codes stored in Redis with a 5-minute TTL, invalidated immediately after first use (replay-proof).

Data Protection



BCrypt with work factor 12 for all password hashing. Card numbers are permanently masked (**** * 1234) — the full PAN is never returned in any API response after storage.

Network Isolation



No direct HTTP calls between microservices — all inter-service communication goes through RabbitMQ. This eliminates lateral movement attack vectors; a compromised service cannot directly call another service's API.

Scalability & Reliability Strategy



Horizontal Scaling

Each microservice can be scaled independently via Docker replicas based on actual load. The Payment Service can run 5× instances during peak hours without touching any other service.



Database Isolation

One database per service ensures a slow query or DB failure in Billing has zero impact on Identity or Payment services. No shared schema, no shared connection pool.



Eventual Consistency

Asynchronous messaging keeps services responsive. If the Notification service is down, RabbitMQ holds the event. The system remains fully functional — emails are delivered once it recovers.



Resiliency via MassTransit

MassTransit provides automatic retry policies with exponential backoff and Dead Letter Queues (DLQ) for messages that exhaust all retries — no message is silently dropped.



Redis Caching

Frequently read data (OTPs, JWT blacklists, session metadata) is served from Redis at sub-millisecond latency, shielding SQL Server from repeated identical queries.



Observability

CorrelationId injected at the Gateway propagates through every service log and event. A single request can be traced end-to-end across all microservices using this shared identifier.

Assumptions & Constraints



Payment Gateway — Simulated

Payment gateway integration (Razorpay / Stripe) is simulated in this version. No actual money is moved. The Saga and risk engine are fully implemented; only the final gateway call is mocked.



Single Currency (INR)

The system assumes all transactions are denominated in Indian Rupees (INR). Multi-currency support requires exchange rate integration and is scoped for a future iteration.



Email as Primary Identifier

User email address is the primary unique identifier across the system. Phone-number-based identity or OAuth SSO (Google, Apple) is out of scope for this version.



Sprint-Scope Constraints

The system is designed within a student-sprint scope, prioritizing correct architectural patterns over full production hardening. Simplified retry logic and basic monitoring are intentional trade-offs acknowledged upfront.

CredVault — High Level Design Document

CredVault · 2026

This document is confidential and intended for internal review only.